

SAE Atelier de développement d'application web
Semestre 4
Sujet Architecture – développement serveur

L'objectif de la SAE est de développer une application web se composant d'un backend gérant des données et proposant des services accessibles pour certains via une interface HTML et pour d'autres via une api retournant des données au format JSON. Le backend est dockerisé. Les données de l'api sont consommées par une application web construite en javascript et par une application mobile construite en flutter. Cette SAE permet de mettre en application des connaissances et des techniques issues de la virtualisation, de l'architecture, de la gestion de données, de la programmation web, de la programmation mobile.

Sujet de travail Architecture : MiniPress.core

L'objectif est de développer MiniPress.core, un mini CMS « headless » qui permet de créer et stocker des articles et de les rendre disponibles en ligne au travers d'une api.

MiniPress.core comprend deux composants principaux :

- Une api mettant à disposition les articles au format JSON. L'api est librement utilisable et permet uniquement de naviguer, rechercher et consulter l'ensemble des articles,
- Une partie Admin permettant de créer des articles en saisissant les données dans des formulaires HTML. Cette partie admin est réservée aux utilisateurs enregistrés.

Les articles gérés par MiniPress sont composés d'un titre, d'un résumé optionnel, d'un contenu, d'une date de création/publication. Des images peuvent être associées à un article sous la forme d'une URL. Les articles sont classés par catégorie. Le résumé et le contenu de l'article sont rédigés en markdown.

L'ensemble de MiniPress.core doit être déployé au sein d'une composition de services docker installés sur la machine `docketu.iutnc.univ.lorraine.fr`. Les deux composants indiqués ci-dessus doivent être exécutés dans deux services séparés.

Fonctionnalités détaillées :

Gestion des articles	1, 2, 3, 4, 5
Gestion des utilisateurs	6, 7, 8
API	9, 10, 11,12
Fonctions étendues	13, 14, 15

Les fonctions étendues sont à réaliser lorsque toutes les autres sont finalisées.

- 1) **Créer un article** : affichage et traitement d'un formulaire de saisie d'un article comprenant le titre, le résumé et le contenu de l'article. La date de création est automatiquement valorisée. Le résumé et le contenu sont saisis et stockés en base de données au format markdown.
- 2) **Créer un article en choisissant une catégorie** : le formulaire de création de l'article permet de choisir sa catégorie parmi une liste de catégories existantes.
- 3) **Afficher la liste des articles** : affichage de la liste des articles sur le CMS, triés par date de création inverse. On affiche uniquement le titre et la date de création de l'article.

- 4) **Afficher la liste des articles en filtrant par catégorie :**
- 5) **Création d'une catégorie :** affichage et traitement d'un formulaire de création d'une catégorie.
- 6) **Formulaire d'authentification :** un utilisateur doit s'authentifier pour pouvoir créer des articles, en fournissant un identifiant (@mail) et un mot de passe.
- 7) **Contrôle d'accès :** seuls les utilisateurs inscrits et authentifiés peuvent créer des articles.
- 8) **Auteurs :** l'utilisateur ayant créé un article est considéré comme son auteur. L'information est disponible avec l'article dans l'affichage de la liste d'utilisateurs.
- 9) **Api :** liste des catégories – la liste des catégories existantes est disponible au format JSON sur l'url `/api/categories`
- 10) **Api :** liste des articles - la liste des articles est disponible au format JSON sur l'url `/api/articles`. Pour un article dans cette liste, seuls ses titres, date de création et auteur sont retournés, accompagnés de l'url permettant d'obtenir l'article complet.
- 11) **Api :** articles d'une catégorie – la liste des articles appartenant à une catégories est disponible au format JSON sur l'url `/api/categories/{id_categ}/articles`. Pour un article dans cette liste, seuls ses titres, date de création et auteur sont retournés, accompagnés de l'url permettant d'obtenir l'article complet.
- 12) **Api :** article complet – un article complet est disponible au format JSON à l'url `/api/articles/{id_a}`
- 13) **Api :** liste des articles d'un auteur – la liste des articles d'un auteur est disponible au format JSON à l'url `/api/auteurs/{id}/articles`,
- 14) **Publication/dépublication des articles :** les articles créés doit être explicitement publiés pour être disponible dans l'api. Il peut être dépublié. La publication/dépublication des articles se fait au travers de l'affichage des listes d'articles.
- 15) **Création d'utilisateurs :** un utilisateur particulier (admin) peut créer de nouveaux utilisateurs ; il saisit l'identifiant et le mot de passe.
- 16) **Tri de la liste d'articles dans l'api :** l'api permet de demander une liste d'articles triée selon la date. Le tri doit être optionnel, et exprimé de la façon suivante :
 - a. `/api/articles?sort=date-asc` : tri selon la date de publication croissante,
 - b. `/api/articles?sort=date-desc` : tri selon la date de publication décroissante
 - c. `/api/articles?sort=auteur` : tri selon l'auteur de l'article

Rendus et évaluation

Le projet doit être rendu au plus tard le mercredi 17 juin 2026 à 16h. Il est attendu, dans l'espace arche prévu à cet effet, un dépôt d'un document pdf comprenant :

- Noms et prénoms des membres du groupe,
- url du dépôt git public contenant le code,
- url de l'application d'administration installée sur `docketu.iutnc.univ-lorraine.fr`
- url de l'api installée sur `docketu.iutnc.univ-lorraine.fr`
- identifiant-mot de passe des utilisateurs créés – au moins deux utilisateurs sont exigés,
- la base de données doit comprendre au moins 8 articles créés dans 3 catégories différentes par au moins deux auteurs distincts. Chaque article doit comprendre un résumé d'au moins 30 mots et un contenu d'au moins 3 paragraphes d'au moins 60 mots chacun. Il est conseillé d'utiliser du Lorem Ipsum.

Démonstration : la démonstration du projet devra montrer la création d'un article qui sera ensuite affiché dans la webapp et dans l'application mobile.

Critères d'évaluation du projet :

Architecture	Respect des principes de découplage des couches	30%	Séparation actions/services/ORM, namespaces et autoloading, répertoires, single action, dispatcher
code	Utilisation de slim/twig/eloquent, qualité du code	30%	Utilisation des fonctionnalités Slim, twig et twig-view, modèles, associations et exception eloquent, application configurable, DRY-KISS, nommage PSR, responsabilité unique
sécurité	csrf, injection, protection des mots de passe	20%	Protection contre l'injection sql et xss, filtrage et validation de données, utilisation d'un token CSRF, protection des mots passés : stockage crypté, contrôle de la qualité
déploiement	Docker compose, docketu	10%	Docker-compose.yml portable et configurable, installation sur docketu
collaboration	Usage de git, branches, tests	10%	Git, branches